Liquid Layering Dispenser

~ Sandor Andrei ~

Theme:

A microprocessor-based device that pours liquids, keeping them separated based on the liquid's density. The device is intended to be used in the making of layered drinks. It can also be used for density demonstrations or dye layering in art applications.

Once the device is plugged in, it raises the poring mount and awaits button input. When the button is pressed, the nozzle lowers and starts pouring. Each liquid has 3 height steps at which the nozzle stops and waits 15 seconds for the liquid to pour, amounting in 45 seconds for each liquid, aprox. 166ml. After the final layer the nozzle lifts up to the starting position and the device awaits input again.

Contents

Liquid Layering Dispenser 1
Theme: 1
Design process: 2
The Prototype: 2
The final design: 3
Components: 3
Code: 6
Theoretical background 10
Conclusions 10
Bibliography 10

Design process:

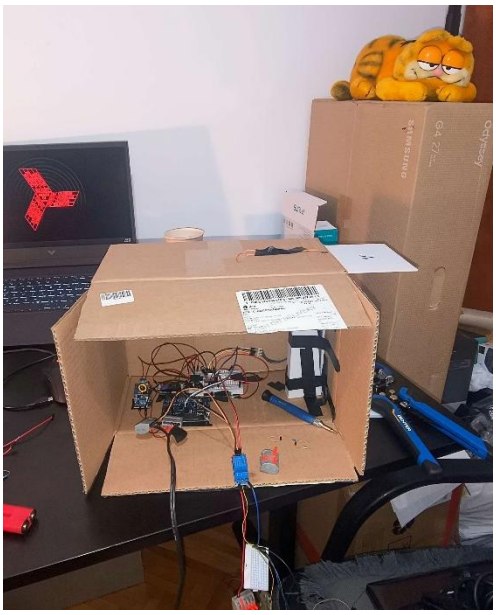
The design process began with a disk that rests on the surface of the liquid, preventing the next poured liquid from breaking the surface tension of the layer below and thereby keeping the layers separated. This was followed by the automation phase, which allows the disk to rise with the liquid level and be removed from the glass once layering is complete.

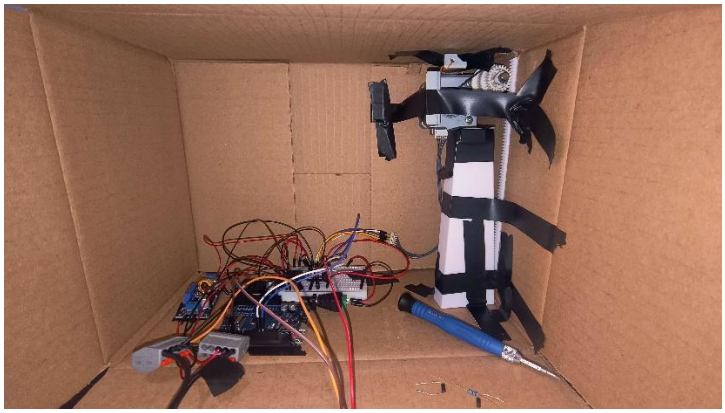
The Prototype:

At first, I thought of moving the disk using a stepper motor with some 3D printed elements attached to the silicone tubing used for the liquids and dispense the liquids using submersible pumps.

During testing, the pumps were found to generate significant electrical noise, adversely affecting the stepper motor's motion.

Additionally, this prototype used a cardboard box as a mounting structure for the components, which lacked sufficient rigidity to keep the motor securely in place and the nozzle mount stable.



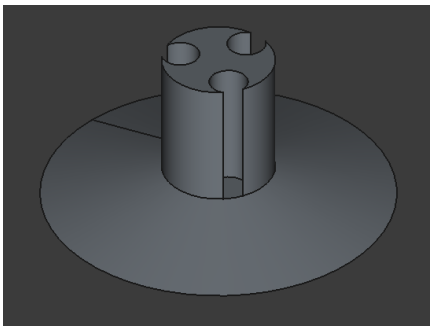


The final design:

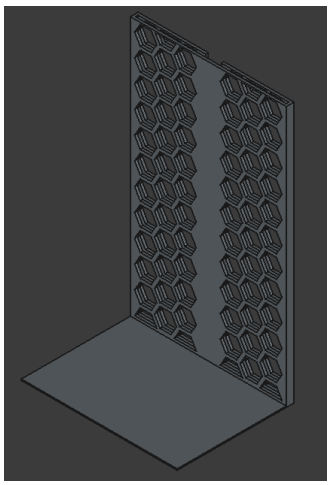
After the prototype I decided to swap the pumps for valves and mount the whole assembly on a wooden frame. The frame also provided enough height for the liquid to flow and storage space for the device's reservoirs.

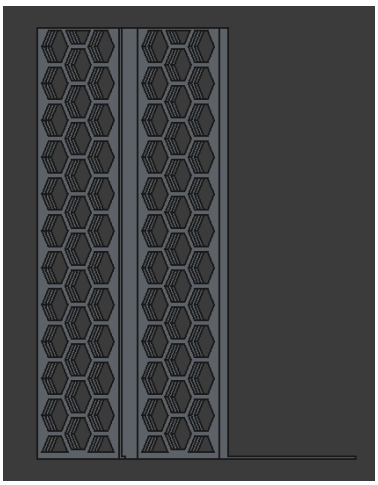
Components:

Nozzle: The nozzle was designed to latch on to the silicone tubing and include the dispersing disk.

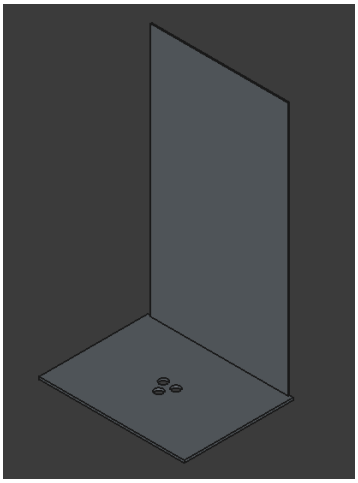


Mount base: The base has a slit to hold and guide the upper part of the mount and also a slit for the rack that allows the stepper to move the uper part.

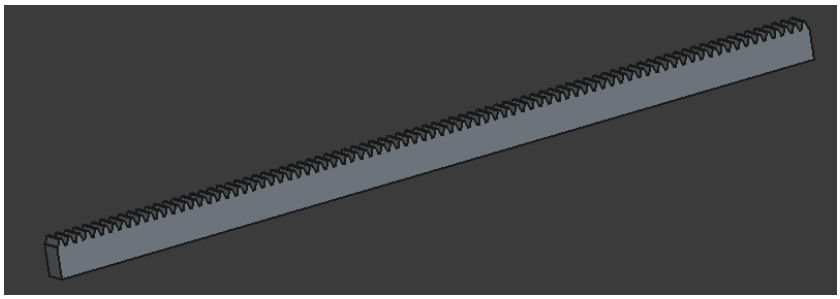




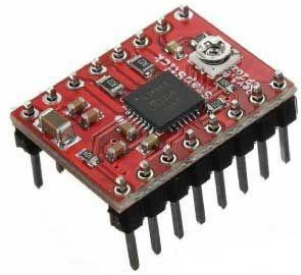
Mount top: The top slides up and down the base and holds the silicone tubing .

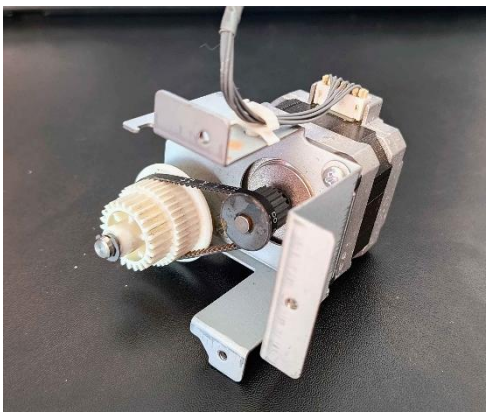


Rack: The rack is glued to the back of the mount top and allows the motor to move the mount.
Its size and number of teeth were designed starting from the diameter of the big cog of the stepper motor

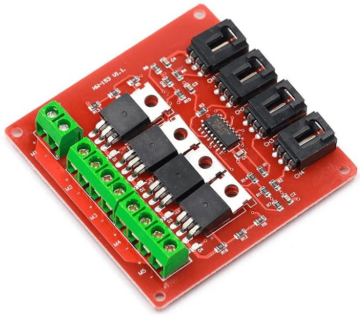


Stepper motor and driver:
A B2234572C 0.3A stepper motor I bought from Piata Oser, that came with a metal mount and two cogs. Its actuated by a A4988 driver.

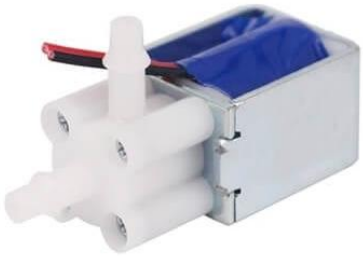




MOSFET module: A 4 MOSFET module used to turn on the pumps using logic signals from the microprocessor and power the 12v electrovalves.

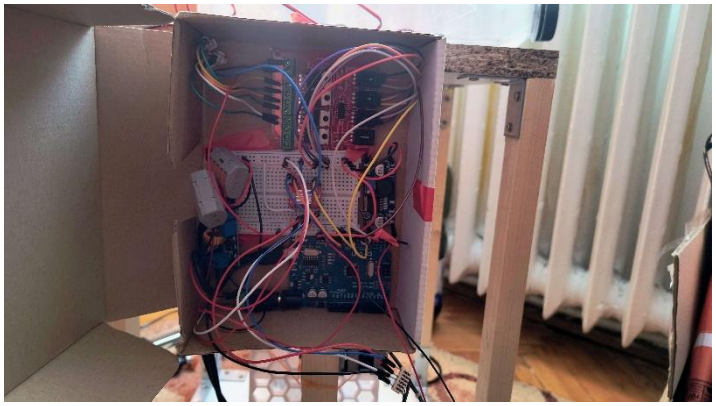


3 x Electrovalves: Allow the liquid from the tanks to flow down the tubing.



Arduino UNO**:** serves as the central microcontroller, managing the device's logic, controlling pumps and motor, and coordinating the timing of the liquid layering process.

Power sources: I used a 15v,4A power source coupled with a buck converter module for 12v to power the valves and another buck converter module for 5v to power the Arduino, the step motor driver, and the MOSFET module.



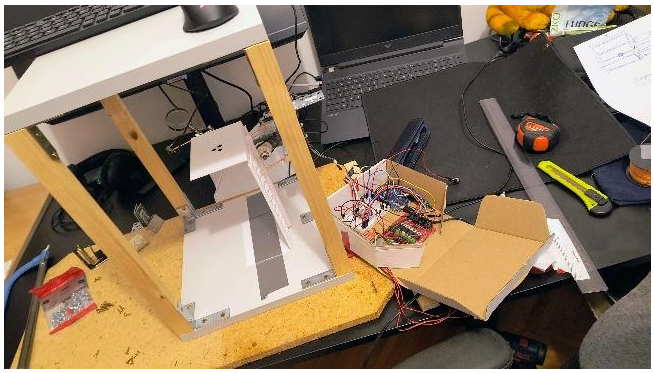
All the electrical components are connected using jumper wires and a breadboard and are attached to a cardboard box acting as housing. The box is attached to the frame using a metal bracket and tape.

The tubing is composed of 3 short (±5cm) silicon tubes from the tanks to one end of the valves and 3 longer tubes (±60cm) from the other end of the valves to the nozzle.

The tanks are made from reused water bottles with some tubing attached.

The wood frame is made of 2 [300x300x20 mm] PAL boards, 5 [20x30x400 mm] wooden elements acting as legs and metal fittings holding it all together.

* Some hardware elements were intentionally secured using tape and string to allow for easy removal, replacement, and future design improvements.



Code:

ASM:

```
	.file	"sketch.cpp"
__SP_H__ = 0x3e
__SP_L__ = 0x3d
__SREG__ = 0x3f
__tmp_reg__ = 0
__zero_reg__ = 1
	.text
	.section	.rodata
	.type	_ZL2SS, @object
	.size	_ZL2SS, 1
_ZL2SS:
	.byte	10
	.type	_ZL4MOSI, @object
	.size	_ZL4MOSI, 1
_ZL4MOSI:
	.byte	11
	.type	_ZL4MISO, @object
	.size	_ZL4MISO, 1
_ZL4MISO:
	.byte	12
	.type	_ZL3SCK, @object
	.size	_ZL3SCK, 1
_ZL3SCK:
	.byte	13
	.type	_ZL3SDA, @object
	.size	_ZL3SDA, 1
_ZL3SDA:
	.byte	18
	.type	_ZL3SCL, @object
	.size	_ZL3SCL, 1
_ZL3SCL:
	.byte	19
	.type	_ZL2A0, @object
	.size	_ZL2A0, 1
_ZL2A0:
	.byte	14
	.type	_ZL2A1, @object
	.size	_ZL2A1, 1
_ZL2A1:
	.byte	15
	.type	_ZL2A2, @object
	.size	_ZL2A2, 1
_ZL2A2:
	.byte	16
	.type	_ZL2A3, @object
	.size	_ZL2A3, 1
_ZL2A3:
	.byte	17
	.type	_ZL2A4, @object
	.size	_ZL2A4, 1
_ZL2A4:
	.byte	18
	.type	_ZL2A5, @object
	.size	_ZL2A5, 1
_ZL2A5:
	.byte	19
	.type	_ZL2A6, @object
	.size	_ZL2A6, 1
_ZL2A6:
	.byte	20
	.type	_ZL2A7, @object
	.size	_ZL2A7, 1
_ZL2A7:
	.byte	21
	.type	_ZL7liquid3, @object
	.size	_ZL7liquid3, 2
_ZL7liquid3:
	.word	7
	.type	_ZL7liquid2, @object
	.size	_ZL7liquid2, 2
_ZL7liquid2:
	.word	6
	.type	_ZL7liquid1, @object
	.size	_ZL7liquid1, 2
_ZL7liquid1:
	.word	5
	.type	_ZL9buttonPin, @object
	.size	_ZL9buttonPin, 2
_ZL9buttonPin:
	.zero	2
	.type	_ZL12directionPin, @object
	.size	_ZL12directionPin, 2
_ZL12directionPin:
	.word	13
	.type	_ZL7stepPin, @object
	.size	_ZL7stepPin, 2
_ZL7stepPin:
	.word	12
	.type	_ZL9stepDelay, @object
	.size	_ZL9stepDelay, 2
_ZL9stepDelay:
	.word	1200
	.type	_ZL18stepsPerRevolution, @object
	.size	_ZL18stepsPerRevolution, 2
_ZL18stepsPerRevolution:
	.word	400
	.type	_ZL11revolutions, @object
	.size	_ZL11revolutions, 2
_ZL11revolutions:
	.word	2
	.type	_ZL10totalSteps, @object
	.size	_ZL10totalSteps, 2
_ZL10totalSteps:
	.word	800
	.type	_ZL12initialSteps, @object
	.size	_ZL12initialSteps, 2
_ZL12initialSteps:
	.word	950
	.type	_ZL10layerSteps, @object
	.size	_ZL10layerSteps, 2
_ZL10layerSteps:
	.word	66
	.type	_ZL11delayUrcare, @object
```

```
        .size
        _ZL11delayUrcare, 2
_ZL11delayUrcare:
        .word   1000
        .type
        _ZL10layerDelay,
@object
        .size
        _ZL10layerDelay, 2
_ZL10layerDelay:
        .word   15000
.global busy
        .section .bss
        .type   busy, @object
        .size   busy, 1
busy:
        .zero   1
        .text
.global _Z6stepUpi
        .type   _Z6stepUpi,
@function
_Z6stepUpi:
        push r28
        push r29
        rcall .
        rcall .
        in r28,__SP_L__
        in r29,__SP_H__
/* prologue: function */
/* frame size = 4 */
/* stack size = 6 */
.L__stack_usage = 6
        std Y+4,r25
        std Y+3,r24
        std Y+2,__zero_reg__
        std Y+1,__zero_reg__
.L3:
        ldd r18,Y+1
        ldd r19,Y+2
        ldd r24,Y+3
        ldd r25,Y+4
        cp r18,r24
        cpc r19,r25
        brge .L4
        ldi r22,lo8(1)
        ldi r24,lo8(12)
        call digitalWrite
        ldi r24,lo8(-80)
        ldi r25,lo8(4)
        call delayMicroseconds
        ldi r22,0
        ldi r24,lo8(12)
        call digitalWrite
        ldi r24,lo8(-80)
        ldi r25,lo8(4)
        call delayMicroseconds
        ldd r24,Y+1
        ldd r25,Y+2
        adiw r24,1
        std Y+2,r25
        std Y+1,r24
        rjmp .L3
.L4:
        nop
/* epilogue start */
        pop __tmp_reg__
        pop __tmp_reg__
        pop __tmp_reg__
        pop __tmp_reg__
        pop r29
        pop r28
        ret
        .size   _Z6stepUpi, .-
_Z6stepUpi
.global setup
        .type   setup,
@function
setup:
        push r28
        push r29
        in r28,__SP_L__
        in r29,__SP_H__
/* prologue: function */
/* frame size = 0 */
/* stack size = 2 */
.L__stack_usage = 2
        ldi r22,lo8(1)
        ldi r24,lo8(7)
        call pinMode
        ldi r22,lo8(1)
        ldi r24,lo8(6)
        call pinMode
        ldi r22,lo8(1)
        ldi r24,lo8(5)
        call pinMode
        ldi r22,0
        ldi r24,lo8(7)
        call digitalWrite
        ldi r22,0
        ldi r24,lo8(6)
        call digitalWrite
        ldi r22,0
        ldi r24,lo8(5)
        call digitalWrite
        ldi r22,lo8(1)
        ldi r24,lo8(13)
        call pinMode
        ldi r22,lo8(1)
        ldi r24,lo8(12)
        call pinMode
        ldi r22,lo8(2)
        ldi r24,0
        call pinMode
        ldi r22,lo8(1)
        ldi r24,lo8(13)
        call digitalWrite
        ldi r24,lo8(-74)
        ldi r25,lo8(3)
        call _Z6stepUpi
        nop
/* epilogue start */
        pop r29
        pop r28
        ret
        .size   setup, .-setup
.global loop
        .type   loop,
@function
loop:
        push r28
        push r29
        rcall .
        rcall .
        rcall .
        in r28,__SP_L__
        in r29,__SP_H__
/* prologue: function */
/* frame size = 6 */
/* stack size = 8 */
.L__stack_usage = 8
        ldi r24,0
        call digitalRead
        or r24,r25
        brne .L7
        lds r25,busy
        ldi r24,lo8(1)
        eor r24,r25
        tst r24
        breq .L7
        ldi r24,lo8(1)
        rjmp .L8
.L7:
        ldi r24,0
.L8:
        tst r24
        brne .+2
        rjmp .L19
        ldi r24,lo8(1)
        sts busy,r24
        ldi r22,0
        ldi r24,lo8(13)
        call digitalWrite
        ldi r24,lo8(-74)
        ldi r25,lo8(3)
        call _Z6stepUpi
        ldi r22,lo8(1)
        ldi r24,lo8(13)
        call digitalWrite
```

```
        ldi r22,lo8(1)
        ldi r24,lo8(5)
        call digitalWrite
        std Y+2,__zero_reg__
        std Y+1,__zero_reg__
.L11:
        ldd r24,Y+1
        ldd r25,Y+2
        sbiw r24,3
        brge .L10
        ldi r22,lo8(-104)
        ldi r23,lo8(58)
        ldi r24,0
        ldi r25,0
        call delay
        ldi r24,lo8(66)
        ldi r25,0
        call _Z6stepUpi
        ldd r24,Y+1
        ldd r25,Y+2
        adiw r24,1
        std Y+2,r25
        std Y+1,r24
        rjmp .L11
.L10:
        ldi r22,0
        ldi r24,lo8(5)
        call digitalWrite
        ldi r22,lo8(1)
        ldi r24,lo8(6)
        call digitalWrite
        std Y+4,__zero_reg__
        std Y+3,__zero_reg__
.L13:
        ldd r24,Y+3
        ldd r25,Y+4
        sbiw r24,3
        brge .L12
        ldi r22,lo8(-104)
        ldi r23,lo8(58)
        ldi r24,0
        ldi r25,0
        call delay
        ldi r24,lo8(66)
        ldi r25,0
        call _Z6stepUpi
        ldd r24,Y+3
        ldd r25,Y+4
        adiw r24,1
        std Y+4,r25
        std Y+3,r24
        rjmp .L13
.L12:
        ldi r22,0
        ldi r24,lo8(6)
        call digitalWrite
        ldi r22,lo8(1)
        ldi r24,lo8(7)
        call digitalWrite
        std Y+6,__zero_reg__
        std Y+5,__zero_reg__
.L15:
        ldd r24,Y+5
        ldd r25,Y+6
        sbiw r24,3
        brge .L14
        ldi r22,lo8(-104)
        ldi r23,lo8(58)
        ldi r24,0
        ldi r25,0
        call delay
        ldi r24,lo8(66)
        ldi r25,0
        call _Z6stepUpi
        ldd r24,Y+5
        ldd r25,Y+6
        adiw r24,1
        std Y+6,r25
        std Y+5,r24
        rjmp .L15
.L14:
        ldi r22,0
        ldi r24,lo8(7)
        call digitalWrite
        ldi r22,lo8(-48)
        ldi r23,lo8(7)
        ldi r24,0
        ldi r25,0
        call delay
        ldi r24,lo8(94)
        ldi r25,lo8(1)
        call _Z6stepUpi
.L18:
        ldi r24,0
        call digitalRead
        ldi r18,lo8(1)
        or r24,r25
        breq .L16
        ldi r18,0
.L16:
        tst r18
        breq .L17
        ldi r22,lo8(10)
        ldi r23,0
        ldi r24,0
        ldi r25,0
        call delay
        rjmp .L18
.L17:
        sts busy,__zero_reg__
.L19:
        nop
/* epilogue start */
        adiw r28,6
        in
__tmp_reg__,__SREG__
        cli
        out __SP_H__,r29
        out
__SREG__,__tmp_reg__
        out __SP_L__,r28
        pop r29
        pop r28
        ret
        .size   loop, .-loop
        .ident  "GCC: (GNU)
7.3.0"
.global __do_copy_data
.global __do_clear_bss
```

More readable C code:

```
const int liquid3 = 7;
const int liquid2 = 6;
const int liquid1 = 5;

const int buttonPin = 0;

const int directionPin = 13;
const int stepPin = 12;

const int stepDelay = 1200;
const int stepsPerRevolution = 400;
const int revolutions = 2;

const int totalSteps = revolutions * stepsPerRevolution;
const int initialSteps = revolutions * stepsPerRevolution + 150;
const int layerSteps = revolutions * stepsPerRevolution / 12;

const int delayUrcare = 1000;
const int layerDelay = 15000;

bool busy = false;

void stepUp(int steps) {
  for (int i = 0; i < steps; i++) {
    digitalWrite(stepPin, HIGH);
    delayMicroseconds(stepDelay);
    digitalWrite(stepPin, LOW);
    delayMicroseconds(stepDelay);
  }
}

void setup() {
  pinMode(liquid3, OUTPUT);
  pinMode(liquid2, OUTPUT);
  pinMode(liquid1, OUTPUT);

  digitalWrite(liquid3, LOW);
  digitalWrite(liquid2, LOW);
  digitalWrite(liquid1, LOW);

  pinMode(directionPin, OUTPUT);
  pinMode(stepPin, OUTPUT);

  pinMode(buttonPin, INPUT_PULLUP);

  digitalWrite(directionPin, HIGH);
  stepUp(initialSteps);
}

void loop() {
  if (digitalRead(buttonPin) == LOW && !busy)
{
    busy = true;

    digitalWrite(directionPin, LOW);
    stepUp(initialSteps);

    digitalWrite(directionPin, HIGH);

    digitalWrite(liquid1, HIGH);
    for (int i = 0; i < 3; i++) {
      delay(layerDelay);
      stepUp(layerSteps);
    }
    digitalWrite(liquid1, LOW);

    digitalWrite(liquid2, HIGH);
    for (int i = 0; i < 3; i++) {
      delay(layerDelay);
      stepUp(layerSteps);
    }
    digitalWrite(liquid2, LOW);

    digitalWrite(liquid3, HIGH);
    for (int i = 0; i < 3; i++) {
      delay(layerDelay);
      stepUp(layerSteps);
    }
    digitalWrite(liquid3, LOW);
    delay(2000);

    stepUp(revolutions * stepsPerRevolution / 4 +
150);

    while (digitalRead(buttonPin) == LOW) {
      delay(10);
    }

    busy = false;
  }
}
```

Theoretical background

The project uses an Arduino Uno to control a stepper motor and dispense liquids automatically. The Arduino reads a button input to start the sequence and controls output pins connected to the valves.

The stepper motor rotates in precise steps, allowing accurate dispensing, while timing functions control the motor speed and dispensing duration. The system tracks whether it is busy to prevent multiple activations at once.

Overall, the theoretical background combines microcontroller programming, digital input/output management, and stepper motor control principles. By coordinating these elements, the system can dispense multiple liquids accurately, safely, and in a repeatable manner, demonstrating the application of embedded systems in automated control tasks.

Conclusions

I consider this project to be an excellent engineering exercise. From conceiving the initial idea to designing, prototyping, experimenting, troubleshooting, and applying theoretical knowledge to a practical, real-world application, the entire process has provided invaluable hands-on experience and insight into the engineering workflow.

While the current design functions effectively, several improvements are possible. These include maintaining consistent reservoir pressure, minimizing tubing length, replacing module-based circuitry with a PCB design, and utilizing lighter frame materials. Despite these potential enhancements, the project successfully demonstrates the application of engineering principles and the current level of technical proficiency.

Bibliography

- https://lastminuteengineers.com/a4988-stepper-motor-driver-arduino-tutorial/
- https://www.youtube.com/watch?v=E14m5hf6Pvo – FreeCAD tutorial